The Math of Logic

Professor Logic's Workbook

One operator. Every formal system. No memorisation required.

James Alexander Pugmire · Propagation Logic Project · 2026

Illustrated by a chalkboard and a great deal of enthusiasm

BEFORE WE BEGIN

A Note from Professor Logic

BEFORE WE BEGIN

A Note from Professor Logic

PROFESSOR LOGIC

Hello. I am very glad you picked this up.

I want to tell you what this book is and what it is not. It is not a logic textbook in the traditional sense. Traditional logic textbooks hand you a list of rules, explain which symbols mean what, and ask you to apply the rules correctly. They treat logic as a given: here are the laws, here is how to use them, here is your grade.

This book does something different. It asks where the laws come from.

The answer turns out to be surprisingly simple. Logical laws are not chosen by committees of philosophers. They are forced out of arithmetic. You pick a collection of values, you pick three operations to work with them, and the laws that hold are the ones the arithmetic makes unavoidable. Change the values and different laws become unavoidable. The mechanism never changes.

That mechanism is what we are going to study. And we are going to study it the only way that actually works: by building things, testing them, and seeing what happens.

There will be a chalkboard. There will be exercises. There will be moments where I ask you to stop and think before reading on. Those moments are not optional extras. They are the actual lesson.

If you work through every exercise, you will finish this book able to look at any formal system in mathematics, logic, or computer science and immediately identify its moving parts. That skill will serve you for the rest of your mathematical life.

One more thing. I have an idea I want to hold onto for later: a distinction operation that might capture the cost of telling two values apart. We will come back to it after we have the foundations in place. For now, just tuck it in the back of your mind.

Right. Let us begin.

CHAPTER 1

What Can Something Be?

The question that starts all of mathematics.

PROFESSOR LOGIC

I want to start with a light switch. Not because light switches are particularly important in the grand scheme of things, but because a light switch is the simplest possible object that has what mathematicians call a value. And value is where everything begins.

A light switch can be on or it can be off. Those are its options. There is nothing else it can be. It cannot be slightly on. It cannot be on-ish. It is either one thing or the other.

Before you do any mathematics with any object, you need to know its options. What can it be? What is allowed? What is not?

The complete collection of things something is allowed to be has a name. I want to write it on the board.

The carrier set (also: value space)

The complete collection of values an object is allowed to take.
Written using curly braces: { } means 'this is a collection'.

Light switch: { ON, OFF } or equivalently { 1, 0 }
Traffic light: { RED, YELLOW, GREEN } or { 0, 0.5, 1 }
Coin: { HEADS, TAILS } or { 0, 1 }
Your age: { 0, 1, 2, 3, ... } -- infinitely many options
Temperature: all real numbers -- the whole number line

PROFESSOR LOGIC

Mathematicians often use numbers instead of words because numbers are easier to calculate with. ON becomes 1. OFF becomes 0. RED becomes 0. YELLOW becomes 0.5. GREEN becomes 1. The meaning depends on the context, but the structure is the same.

I used 0.5 for yellow. Why? Because yellow is in the middle: not fully stopped, not fully going. Halfway.

And 0.5 is halfway between 0 and 1. That is not a coincidence. The numbers are chosen to reflect the structure of the options.

ASK YOURSELF

Before reading on: what is the carrier set for each of the following? Write them out using curly braces. 1. A dice. 2. A yes or no question. 3. The grade on an exam (A, B, C, D, F). 4. A compass heading (rough: North, South, East, West).

Designated values

PROFESSOR LOGIC

Here is a subtlety that matters a great deal later. In most carrier sets, some values count as true or holding and others do not. In a logic where the carrier is {0, 1}, the value 1 is the designated value: it represents true, correct, happening. The value 0 is not designated.

This might seem obvious for a two-value system. It becomes interesting when you have more values. In the traffic light carrier {0, 0.5, 1}, which values are designated? It depends on what question you are asking. If the question is should I go, only 1 is designated. If the question is should I be alert, perhaps 0.5 and 1 are both designated.

The designation rule is a choice you make. Different choices give different systems.

Designated value

A value in the carrier that counts as 'true' or 'holding.'

In classical logic: carrier is {0, 1} and the single designated value is 1.
In a three-valued carrier: there may be multiple designated values.

ASK YOURSELF

In the carrier {0, 0.5, 1} with the designation rule 'any value above zero is designated': Which values are designated? Which are not? How many designated values are there?

CHAPTER 1 EXERCISES

1.1 Write the carrier set for each situation. Justify your encoding choice.

(a) A light with three settings: off, dim, bright.

(b) A sensor reading soil moisture as a percentage.

(c) A database field that can hold yes, no, or "not yet recorded".

Answer:

(a) {0, 0.5, 1}: off=0, dim=0.5, bright=1. Three evenly spaced values, reflecting equal steps in brightness.

(b) [0, 1]: continuous range representing percentages.

(c) {0, N, 1} where 0=no, 1=yes, N=not recorded. N is a third value that is neither yes nor no.

1.2 A student argues: "All carrier sets can be written using only 0 and 1." Are they right?

Give an example that supports their view and an example that defeats it.

Answer:

They are right for two-valued systems: any two options can be encoded as 0 and 1.

They are wrong for continuous systems: the carrier [0,1] contains infinitely many values that cannot all be encoded as just 0 and 1.

□□ 1.3 Design a carrier for a court verdict system needing: guilty, not guilty, and verdict still pending.

State your carrier, your encoding, and your designation rule.

Explain what it would mean to change the designation rule.

Answer:

Carrier: {0, 0.5, 1} where 0=not guilty, 0.5=pending, 1=guilty.

Designation rule: only 1 (guilty) is designated.

Alternative: {0.5, 1} both designated, meaning cases not yet resolved.

Changing the designation rule changes which values trigger consequences. The carrier itself stays the same.

□□□ 1.4 Can a carrier have zero elements? Can it have exactly one element?

What happens to logic in each case?

Answer:

Zero elements: impossible as a logical carrier. With no values, no statement can be evaluated. The system cannot say anything.

One element: possible but produces trivial logic. If $V = \{1\}$, then every statement evaluates to 1. Nothing can be false. Useless for reasoning about anything that could be false.

CHAPTER 2

The Three Operations

The tools you use to work with any carrier.

PROFESSOR LOGIC

A carrier set by itself does not do anything. It just sits there, holding its options. To reason with values, you need operations: rules for taking one or more values and producing a new value. Three operations appear in essentially every logic system ever built. They are called NOT, AND, and OR. I want to introduce them in a way that makes their formulas feel inevitable rather than arbitrary. Because they are inevitable. Once you choose your carrier, the formulas for these operations are forced.

NOT: the flip machine

PROFESSOR LOGIC

NOT takes one value and gives back its opposite. In the light switch carrier: NOT(ON) = OFF, NOT(OFF) = ON. NOT is a reversal machine. You put something in, you get the other thing out.

NOT

NOT(v) = 1 - v

Check:
NOT(1) = 1 - 1 = 0 (true becomes false)
NOT(0) = 1 - 0 = 1 (false becomes true)

In the three-value carrier {0, 0.5, 1}:
NOT(0.5) = 1 - 0.5 = 0.5 (the middle value is its own opposite)

Notice: the value 0.5 flips to itself under NOT. This will become very important in Chapter 7.

ASK YOURSELF

Compute NOT(NOT(v)) for v = 0, v = 0.5, and v = 1. What pattern do you see? Write it as a general rule before reading on.

PROFESSOR LOGIC

Did you get it? Flipping twice returns you to where you started. NOT(NOT(v)) = v. Always. This is called Double Negation, and we will prove it holds for every value in Chapter 3. For now, just notice that it is a consequence of the formula 1 minus v, not a rule we added separately.

AND: the strict gatekeeper

PROFESSOR LOGIC

AND takes two values and combines them. The result is designated only when both inputs are designated. Imagine a security door that requires both a keycard AND a PIN. One alone is not enough. Both, or nothing.

AND

AND(a, b) = a x b (multiplication)

Truth table for {0, 1}:
AND(0, 0) = 0 x 0 = 0
AND(0, 1) = 0 x 1 = 0
AND(1, 0) = 1 x 0 = 0
AND(1, 1) = 1 x 1 = 1 <- only this row gives 1

Why multiplication? The only way to get 1 x 1 = 1 is when both are 1.
Any zero in the input gives a zero output. The strict gatekeeper.

OR: the generous one

OR

OR(a, b) = max(a, b) (the larger of the two)

Truth table for {0, 1}:
OR(0, 0) = max(0, 0) = 0
OR(0, 1) = max(0, 1) = 1
OR(1, 0) = max(1, 0) = 1
OR(1, 1) = max(1, 1) = 1 <- fails only when both are 0

Why maximum? The designated value (1) is the largest.
As long as one input is 1, the maximum is 1.

PROFESSOR LOGIC

The formulas for AND and OR are not arbitrary choices. They follow from what AND and OR are supposed to mean, combined with the decision to use 0 and 1 as the values. Given those two decisions, the formulas are the natural result. Multiplication gives 'both must be true.' Maximum gives 'at least one must be true.' You did not need to memorise them. You could derive them.

Load: operations have costs

PROFESSOR LOGIC

Every operation has a cost. We will explore this fully in Chapter 5, but I want to plant the seed now. NOT is cheap: you just flip the value. AND is expensive: it adds the costs of both inputs together. OR is clever: it routes through the cheaper input. This will matter when we encounter paradoxes. For now: remember that operations are not free.

ASK YOURSELF

Compute the following. Show your working. 1. NOT(NOT(0.5)) 2. AND(0.5, NOT(0.5)) 3. OR(0.5, NOT(0.5)) 4. AND(NOT(1), NOT(0))

CHAPTER 2 EXERCISES

□ 2.1 Build a complete truth table for NOT, AND, and OR on the carrier {0, 1}.

Four rows for AND and OR. Two rows for NOT.

Answer:

NOT: NOT(0)=1, NOT(1)=0.

AND: (0,0)=0, (0,1)=0, (1,0)=0, (1,1)=1.

OR: (0,0)=0, (0,1)=1, (1,0)=1, (1,1)=1.

□ 2.2 Compute in {0, 0.5, 1} using NOT(v)=1-v, AND=multiply, OR=max:

(a) NOT(0.5) (b) AND(0.5, 0.5) (c) OR(0.5, 0.5) (d) NOT(AND(1, 0.5))

Answer:

(a) 0.5 (b) 0.25 (c) 0.5 (d) NOT(0.5) = 0.5

□□ 2.3 Show that A -> B = OR(NOT(A), B). Test all four input combinations.

Implication equals 0 only when A=1 and B=0; in all other cases it equals 1.

Answer:

OR(NOT(A), B): (A=0,B=0): OR(1,0)=1 (A=0,B=1): OR(1,1)=1 (A=1,B=0): OR(0,0)=0 (A=1,B=1): OR(0,1)=1 .

Implication is derivable from NOT and OR.

□□ 2.4 Why does OR use maximum rather than addition?

Hint: what would OR(1, 1) = 1 + 1 = 2 mean in a {0, 1} carrier?

Answer:

Addition gives OR(1,1) = 2, which is outside the carrier {0,1}. An operation must map carrier values to carrier values (closure). Addition violates closure. Maximum is correct because it always stays within the carrier.

CHAPTER 3

Laws That Cannot Break

Some truths are forced. Others are merely true for now.

PROFESSOR LOGIC

Here is a question I want you to sit with for a moment. Is the law 'a thing cannot be both true and false at the same time' something someone chose? A rule a philosopher decided to include? Or is it something that falls out of the arithmetic of {0, 1}? The answer is the second one. And that distinction matters enormously. Because a rule that someone chose could be different if they had chosen otherwise. But a consequence of arithmetic cannot be different. It is forced.

Forced law

A statement is a forced law of a carrier if it is true
for every possible value in that carrier.

How to check: test every value. One failure is enough to disprove.
No failures across all values: the law is forced.

The Law of Non-Contradiction

LNC: AND(v, NOT(v)) = 0 for all v in V

Test in {0, 1}:
v = 0: NOT(0) = 1. AND(0, 1) = 0 x 1 = 0. □
v = 1: NOT(1) = 0. AND(1, 0) = 1 x 0 = 0. □
Both values checked. Both passed. LNC is forced in {0, 1}.

Now test in {0, 0.5, 1}:
v = 0: AND(0, 1) = 0. □
v = 1: AND(1, 0) = 0. □
v = 0.5: NOT(0.5) = 0.5. AND(0.5, 0.5) = 0.25. □ 0.25 is not 0

One failure. LNC is not forced in {0, 0.5, 1}.

PROFESSOR LOGIC

Same formula. Same operations. Different carrier. Different result. The law did not change. The arithmetic changed. This is what it means to say the laws are forced by the carrier, not chosen by us.

The Law of Excluded Middle

LEM: OR(v, NOT(v)) = max(V) for all v in V

In {0,1}: max(V) = 1. In {0, 0.5, 1}: max(V) = 1.

Test in {0, 1}:
v = 0: OR(0, 1) = 1. □
v = 1: OR(1, 0) = 1. □
LEM is forced in {0, 1}.

Test in {0, 0.5, 1}:
v = 0.5: NOT(0.5)=0.5. OR(0.5, 0.5) = 0.5. 0.5 is not 1. □
LEM is not forced in {0, 0.5, 1}.

Double Negation

DN: NOT(NOT(v)) = v for all v in V

For NOT(v) = 1 - v:
NOT(NOT(v)) = 1 - (1 - v) = v Always.

This is pure algebra. It holds for any value in any carrier that uses NOT(v) = 1-v.
It does not require checking case by case. The formula proves it directly.

The master table

Laws across carriers

Law {0,1} Classical {0,0.5,1} [0,1] Fuzzy
Non-Contradiction forced fails at 0.5 fails at interior
Excluded Middle forced fails at 0.5 fails at interior
Double Negation forced forced forced

Notice: Double Negation is the most robust law.
It survives carrier changes that destroy LNC and LEM.
This is because it depends only on the formula for NOT,
not on AND or OR.

"The laws are not carved in stone. They are printed on carrier paper. Change the carrier and different laws come with it."

CHAPTER 3 EXERCISES

3.1 Check whether LNC holds in {0, 0.33, 0.67, 1} using AND=multiply and NOT(v)=1-v.

Test every value. State which pass and which fail.

Answer:

v=0: AND(0,1)=0 . v=0.33: NOT(0.33)=0.67, AND(0.33,0.67)=0.221 not 0 .
v=0.67: AND(0.67,0.33)=0.221 not 0 . v=1: AND(1,0)=0 .
LNC fails at 0.33 and 0.67.

□□ 3.2 A student claims: if LNC fails in a carrier, LEM must also fail. Test this claim.

Find a carrier where LNC fails but LEM holds, or prove no such carrier can exist.

Answer:

Using $V = \{0, 0.5, 1\}$ with AND = min instead of multiply: LNC fails at 0.5 (min(0.5,0.5)=0.5 not 0). LEM also fails (max(0.5,0.5)=0.5 not 1). With asymmetric NOT where NOT(0.5)=0: LNC holds at 0.5 (AND(0.5,0)=0) but LEM fails (OR(0.5,0)=0.5 not 1). So LNC holding does not force LEM.

□□ 3.3 Prove that Double Negation holds for ANY carrier, as long as NOT(v) = 1-v.

Hint: use algebra rather than testing individual values.

Answer:

NOT(NOT(v)) = NOT(1-v) = 1-(1-v) = 1-1+v = v. This algebraic identity holds for any real number v, so it holds for any carrier whose values are real numbers and whose NOT uses the formula 1-v. QED.

□□□ 3.4 Is it possible to design a carrier where LNC holds but Double Negation fails?

If yes, construct one. If no, explain why not.

Answer:

Yes. Define $V = \{0, 1\}$ and NOT(0) = 0, NOT(1) = 0. Then NOT(NOT(1)) = NOT(0) = 0, not 1. DN fails at v=1. Check LNC: AND(0,NOT(0))=AND(0,0)=0 . AND(1,NOT(1))=AND(1,0)=0 . LNC holds. So LNC can hold while DN fails, if we use a non-standard NOT.

CHAPTER 4

Three Knobs

Every formal system is completely specified by exactly three things.

PROFESSOR LOGIC

Here is something that will rearrange how you think about logic, mathematics, and formal systems of all kinds. Every formal system ever built is completely specified by three parameters. Three knobs on a machine. Turn one knob and some laws change. The others stay. Turn a different knob and a different set of laws changes. The mechanism never changes. Only the outputs do.

The three parameters

V -- the value carrier (what values are possible)
G -- the gradient family (which operations are available)
theta -- the coherence threshold (how much load the system tolerates)

Change V: different laws become forced.
Change G: some laws become unreachable.
Change theta: different patterns can cohere.

Knob 1: the value carrier (V)

PROFESSOR LOGIC

We have already seen this one in action. Changing V from {0, 1} to {0, 0.5, 1} broke LNC and LEM but left Double Negation intact. The operations were identical. Only the carrier changed.

What changing V does

Classical: V = {0, 1} LNC forced, LEM forced, Liar is paradox
Three-valued: V = {0, 0.5, 1} LNC fails, LEM fails, Liar resolves at 0.5
Fuzzy: V = [0, 1] LNC fails everywhere, Liar resolves at 0.5
Calculus: V = all real numbers Different laws entirely (product rule, chain rule)
Probability: V = [0,1] normalised Kolmogorov's axioms forced

Where the binary carrier comes from

NEW INSIGHT: A = A GENERATES THE BINARY CARRIER

The binary carrier $V = \{0, 1\}$ feels like the obvious, natural default. It is not.

It is a specific choice forced by three hidden assumptions.

When you write 'A equals A' you are making three invisible claims:

Assumption 1: Absoluteness. A thing either is A or it is not. No degrees.

This rules out values like 0.3 and forces the binary choice: 0 or 1.

Assumption 2: Temporal Stability. What is A at time t_1 is A at time t_2.

This rules out time-indexed value spaces.

Assumption 3: Context Independence. What is A in context C_1 is A in context C_2.

This forces the load L to be zero -- the pattern must be valid in all contexts.

All three assumptions together force $V = \{0, 1\}$.

And once you have $V = \{0, 1\}$, LNC, LEM, and DN follow automatically.

The axioms are not chosen. They fall out.

What each denial produces

Deny Absoluteness Quantum mechanics: superposition -- multiple values simultaneously
Deny Temporal Stability Dynamics: A(t1) = A(t2) is a non-trivial claim needing a mechanism
Deny Context Independence Relativity and economics: value depends on the observer

Classical logic holds all three simultaneously.
Most real domains violate at least one.
Knowing which assumption is being made is the formal system engineer's job.

Knob 2: the gradient family (G)

PROFESSOR LOGIC

The gradient family is the set of operations available in the system. We have been working with {NOT, AND, OR}. But you can restrict this. And when you do, some laws become unreachable: not false, just impossible to state.

Intuitionistic logic: OR removed from G

Classical logic: G = {NOT, AND, OR, ->}
Intuitionistic: G = {NOT, AND, ->} (OR removed)

LEM says: OR(v, NOT(v)) = 1
Without OR in G, this statement cannot even be formed.
LEM is not false in intuitionistic logic. It is unreachable.

LNC still holds (uses AND, which is still in G).
The system is not broken. It is more restricted. More careful.

Knob 3: the coherence threshold (theta)

PROFESSOR LOGIC

Every time you apply an operation, the result carries some cost. A pattern with a cost above theta cannot cohere in the system. We will explore this fully in Chapter 5. For now: theta is the third knob, and it controls how complex a pattern can be before the system can no longer hold it. Think of it as a circuit breaker. Too much load, and the circuit opens.

ASK YOURSELF

Without doing any calculation: if I lower theta from 1.0 to 0.5, what do you predict will happen? Will there be more or fewer patterns that can cohere?

CHAPTER 4 EXERCISES

4.1 For each change below, state which laws change and which stay the same.

(a) Change V from {0,1} to {0, 0.5, 1}.

(b) Remove OR from G.

(c) Double theta from 1.0 to 2.0.

Answer:

(a) LNC and LEM break. DN survives.

(b) LEM becomes unreachable (not false, unreachable). LNC still holds.

(c) More patterns can cohere. No logical laws change. The system becomes more permissive about complex patterns.

□□ 4.2 Name the three hidden assumptions in A = A. For each, name the formal system produced by denying it.

Answer:

Absoluteness -> denied -> quantum mechanics (superposition).

Temporal Stability -> denied -> dynamics and conservation laws.

Context Independence -> denied -> special relativity and context-sensitive systems.

□□□ 4.3 Can you change all three parameters simultaneously and still recognise the result as a logic?

What is the minimum requirement for something to deserve the name?

Answer:

Yes. The minimum requirements are: (1) V is non-empty. (2) Operations in G are closed (produce values in V). (3) theta is a positive real number. As long as these hold, you have a consistent formal system. Whether it is useful is a separate question.

CHAPTER 5

The Price of an Idea

Every distinction costs something. The universe insists on this.

PROFESSOR LOGIC

In 1961, a physicist named Rolf Landauer published a paper arguing that erasing one bit of information requires a minimum amount of energy. Not as a practical matter, but as a matter of physics. The thermodynamic cost of destroying a distinction between two states cannot be zero.

The Landauer limit

L_min = kB x T x ln(2) approximately 2.87 x 10^-21 joules

kB = Boltzmann's constant (1.38 x 10^-23 J/K)
T = temperature in Kelvin (room temperature is 300K)
ln(2) is approximately 0.693

This is called the Landauer limit.
It is a physical floor, not an engineering limitation.
Verified in laboratory experiments to within 2% (Berut et al., 2012).

PROFESSOR LOGIC

Why does this matter for logic? Every logical operation is, at its core, a manipulation of distinctions. True and false are distinct. Maintaining, combining, and erasing these distinctions requires work. The universe charges for this service. We can abstract the cost as a dimensionless number called load, and we track it through every operation.

Load combination rules

A pattern is written as P = (value, load)

NOT(P): value = 1 - v load = L (unchanged: just a flip)
AND(P,Q): value = vP x vQ load = LP + LQ (loads add: expensive)
OR(P,Q): value = max(vP,vQ) load = min(LP, LQ) (takes cheaper path)

PROFESSOR LOGIC

AND is expensive because it requires both patterns to hold simultaneously. The system must maintain both distinctions at once. Their costs add. OR is clever: it only needs one of the two patterns, so it routes through the cheaper one. The costly one can be ignored.

NEW INSIGHT: THE GENERALISED AND RULE

The AND load rule L = LP + LQ assumes that P and Q are independent.

That is the right assumption when your evidence is genuinely independent.

When evidence is correlated -- when knowing P tells you something about Q --

the true cost of maintaining both is less than LP + LQ.

You get a discount proportional to the mutual information I(P, Q).

The generalised rule is: AND load = LP + LQ - I(P, Q)

Independent evidence: I(P,Q) = 0. Standard rule. Pay full price.

Perfectly correlated: I(P,Q) = LP. Q is free given P. Pay for one.

Practical consequence: ten studies using the same methodology do not

give you ten independent pieces of evidence. They share information.

Their true joint load is much less than 10 x the unit load.

The standard AND rule would make them look like ten. It is not honest.

NEW INSIGHT: THE FULL LOAD SPECTRUM

The standard view treats load as L >= 0. This is correct for contingent claims -- claims that could be otherwise. But there is an extension.

A tautology (a statement true in every state of the carrier) has no adversarial context. There is nothing to maintain the distinction against. Its load should be sub-zero: it earns stability rather than spending it. The Landauer floor applies only to contingent claims.

L = -inf Logical contradiction. Cannot cohere in any context.

L < 0 Tautology. Earns stability. Coheres even in restrictive contexts.

L = 0 Seed state: absolute, context-free, no history.

0 < L < theta Coherent contingent claim. Within the context capacity.

L = theta At the coherence boundary.

L > theta Incoherent in this context. Requires a larger context.

L = +inf Mode 2 paradox. No finite context can absorb it.

The coherence threshold

theta = maximum load a pattern can carry and still cohere

If load(P) <= theta --> P is coherent. Can be used in further reasoning.
If load(P) > theta --> P is incoherent. The system cannot hold it.

Example: theta = 10.
AND((1, 4), (1, 3)) = (1, 7) coherent (7 <= 10)
AND((1, 7), (1, 5)) = (1, 12) incoherent (12 > 10)

DRAS: carrying your history

PROFESSOR LOGIC

The fine structure constant of physics is approximately 1/137 at low energies and 1/129 at high energies. It changes with scale. It is not constant at all. The De-Reification Axiom Standard (DRAS) applies this insight to formal systems. Every quantity should carry its full history, including the scale at which it was measured. Writing just '1/137' without specifying the energy scale is a reification: treating a context-dependent measurement as if it were context-free. Every pattern P = (value, load) is the DRAS version of a logical statement. The value is what you would normally write. The load is the hidden cost you would normally ignore. Carrying both is more honest.

ASK YOURSELF

A proof has seven AND operations in sequence. Each input pattern has a load of 2.0. What is the load after three AND operations? After seven? If theta = 25, does the proof cohere? At which step does it first fail?

CHAPTER 5 EXERCISES

5.1 Compute the value and load of each result. theta = 10. State whether each coheres.

(a) NOT((1, 4.0)) (b) AND((1, 3.0), (0, 5.0)) (c) OR((1, 8.0), (1, 3.0))

Answer:

(a) NOT: value=0, load=4.0. Coherent.

(b) AND: value=0, load=8.0. Coherent.

(c) OR: value=1, load=min(8,3)=3.0. Coherent.

5.2 Why is OR cheaper than AND? Explain in plain language, then connect to the load formulas.

Answer:

OR needs only one of its inputs to be true, so it takes the minimum of the two loads: the cheaper path. AND needs both inputs simultaneously, so both costs must be paid. In plain language: "either this OR that" is easier to maintain than "this AND that."

5.3 What is the load of a tautology? Explain why the Landauer floor does not apply to it.

Answer:

A tautology has $L < 0$. It is true in every state of the carrier. There is no adversarial context -- nothing to maintain the distinction against. The Landauer floor (kT ln2 per bit erased) applies to erasing contingent information. A tautology cannot be otherwise, so there is nothing to erase.

5.4 A student says: "If we set theta = infinity, every proof will cohere, so we will get a complete logic." Is this correct?

Answer:

The student is partially right: with theta = infinity, no pattern is ever rejected for load reasons. But some true statements require self-referential proofs whose load grows without bound (2^depth). Even at theta = infinity, you would need infinitely many reasoning steps for some statements. The problem is not the threshold; it is the infinite load itself. DRAS reframes this as a thermodynamic observation: some truths cost infinite work to establish.

CHAPTER 6

The Debt of the Distinction

Every boundary costs something. Ignore that cost long enough and the universe sends a bill.

PROFESSOR LOGIC

In the last chapter we met Landauer's principle. But there is a deeper consequence I want to sit with now. Not just the cost of erasing a distinction. The cost of maintaining one. A boundary between true and false is not a line drawn in ink. It is an active physical process. Something must hold those two states apart. In biology, a cell membrane maintains the distinction between inside and outside at continuous thermodynamic cost. In computing, a flip-flop maintains the distinction between 0 and 1 at continuous power consumption. Classical logic does not charge for this maintenance. It treats all distinctions as free. That is a debt, not a free lunch.

What a boundary actually is

```
Classical logic:
    TRUE | FALSE
         ^
    This line is assumed to be infinitely sharp and free to maintain.
In reality:
    TRUE [gradient zone] FALSE
              ^
    This region costs energy to resolve.
    The closer a state is to the boundary, the more it costs to classify.
```

The 0.5 test

NEW INSIGHT: THE 0.5 TEST FOR REIFICATION

There is a simple diagnostic for whether you are applying the wrong carrier.

For any binary claim, ask: what would the value 0.5 mean?

If the answer is clear and sensible: the domain is continuous and you are applying a binary carrier to it. That is reification. The distinction debt is real.

If 0.5 is incoherent for this domain: the binary carrier may actually fit.

Examples:

'He is intelligent or not' -- 0.5 = moderately intelligent. Meaningful. Continuous.

'The contract is valid or invalid' -- half-valid is mostly incoherent. Binary may fit.

'She is guilty or not guilty' -- 0.5 = somewhat guilty has meaning in ethics but not in law. The court imposes binary on a continuous domain.

'The circuit is open or closed' -- 0.5 = half-open is physically incoherent. Binary fits.

When 0.5 makes sense, the distinction debt is real and accumulating.

PROFESSOR LOGIC

Consider a medical test that returns POSITIVE or NEGATIVE. The test is measuring a continuous biological quantity. At some threshold, a line has been drawn. That line is a classical {0,1} distinction imposed on a [0,1] physical reality. The distinction is useful. But it is not free. Every case near the threshold pays the debt as false positives and false negatives. The logic said 'this is a crisp boundary.' The biology did not get the memo.

Types of distinction debt

Precision debt:

Claiming more certainty than the system actually has.
The logic returns TRUE or FALSE. The measurement was 0.51.
Cost: false confidence. Errors that look like correct results.

Boundary debt:

Maintaining a sharp line where nature has a gradient.
Cost: edge cases. Exceptions. The category 'other.'

Accumulation debt:

Each AND operation adds loads. Classical logic does not track this.
Cost: reasoning chains that seem valid but exceed the context's capacity.

PROFESSOR LOGIC

I want to be clear about something important. Classical logic is not wrong. It is a pragmatic bargain. You agree to treat all distinctions as free and sharp, and in return you get a powerful, tractable formal system. The bargain is worth making in many contexts. The problem is when the bargain is made without acknowledgement. When a binary logic is applied to a continuous problem and the hidden costs are called 'edge cases' rather than 'the distinction debt we forgot to account for.' The debt does not disappear when it is unacknowledged. It compounds.

Newton did not fail. Einstein found his theta.

NEW INSIGHT: CARRIER-RELATIVE LOAD AND SCIENTIFIC REVISION

Newtonian mechanics is a carrier. Within it, the velocity addition rule

v_total = v1 + v2 is not a hypothesis. It is a consequence of the carrier structure.

Its load within that carrier is zero: it is free to maintain.

Einstein's relativity enriches the carrier. In the relativistic carrier, the

Newtonian velocity rule has load approximately equal to (v/c) squared.

At v = 0.001c: load is roughly one part in a million. Negligible.

At v = 0.5c: load is 25%. Significant.

At v = 0.99c: load exceeds 98%. Completely wrong.

Newton's mechanics was not wrong.

It was a carrier with a hidden coherence threshold: v much less than c.

Einstein did not refute Newton. He found Newton's theta.

This is the correct structure of all scientific revision.

The old theory is not false. It is valid within a domain it could not name.

The new theory reveals the domain conditions.

Every paradigm shift has this structure.

CHAPTER 6 EXERCISES

6.1 A speed camera classifies as SPEEDING or NOT SPEEDING at 30 mph. Measurement uncertainty is plus or minus 2 m

Answer:

The measurement has uncertainty of 2 mph. The true speed is in [29, 33] mph. The sharp threshold cannot distinguish a car doing 31 from one doing 29 within measurement error. DRAS would require carrying the measurement as (31 mph, 2 mph uncertainty). The threshold decision should reflect the uncertainty: a car at 31 plus or minus 2 is borderline.

6.2 Apply the 0.5 test to each claim. State whether the binary carrier fits or is reified.

(a) "The patient is ill." (b) "The switch is off." (c) "The economy is failing." (d) "2 + 2 = 4 is true."

Answer:

(a) 0.5 = moderately ill is meaningful. Continuous domain. Binary reified.

(b) 0.5 = half-off is physically incoherent for a toggle switch. Binary fits.

(c) 0.5 = somewhat failing is meaningful. Continuous domain. Binary reified.

(d) 0.5 = somewhat true is incoherent for a mathematical statement. Binary fits.

6.3 Describe a situation in your own experience where a classical {0,1} distinction is applied to a genuinely continuou

Answer:

Open question. Look for: binary grades on continuous performance. Pass/fail exams on continuous understanding. Legal guilty/not guilty on continuous evidence. Poverty lines on continuous income. Any binary policy on continuous human need.

CHAPTER 7

When Systems Break

Four modes. Each one a different kind of overload.

PROFESSOR LOGIC

A paradox is not a mystery waiting for a philosophical answer. A paradox is a gradient overload. The system demanded a cost it could not pay. There are exactly four ways this can happen. I want to show you all four, and I want to show you that each one is a distinct failure mode with a distinct structure. Two thousand years of philosophy has been, in part, a conversation about these four modes without having the vocabulary to distinguish them.

Mode 1: The Liar (no fixed point)

This sentence is false.

Let v = truth value of the sentence.
The sentence demands: v = NOT(v)

Test in {0, 1}:
v = 0: NOT(0) = 1 is not 0. □
v = 1: NOT(1) = 0 is not 1. □
No value satisfies v = NOT(v). The equation has no solution in {0,1}.

In {0, 0.5, 1}:
v = 0.5: NOT(0.5) = 0.5 = v. □ Resolved.

The Liar is not a mystery. It is the carrier {0,1} not containing 0.5.

PROFESSOR LOGIC

The Liar paradox is two thousand years old. Epimenides of Crete said 'all Cretans are liars' (he was Cretan). Philosophers have argued about it ever since. The argument dissolves the moment you look at it as an equation in a value space. The equation v = NOT(v) needs v = 0.5. If the value space does not contain 0.5, there is no solution. Not 'no philosophical answer.' No arithmetic solution. Different thing entirely.

Mode 2: Load diverges (Godel, Turing)

Mode 2: the load grows without bound

A self-referential proof requires its own proof as a premise.
Depth 1: load = 1
Depth 2: load = 2 (references depth 1)
Depth 3: load = 4 (references depth 2)
Depth n: load = 2^n

For any finite theta, there exists a depth n where 2^n > theta.
The proof cannot cohere. Not because it is false. Because it costs too much.

Godel's incompleteness: every sufficiently powerful system contains true statements that are too expensive to prove within the system.
This is the Mode 2 explanation.

Mode 3: Level collapse (Russell)

PROFESSOR LOGIC

Bertrand Russell, in 1901, discovered that naive set theory allows the set R = {x : x is not a member of itself}. Ask whether R is a member of itself and the same flip-flop appears. But this is structurally different from the Liar. There is no infinite load accumulation. Instead, the gradient family demands a context one level larger than the one available. The system needs to look at itself from outside itself. It cannot.

Mode 3: the gradient demands a larger context

Russell's set R: is R a member of R?
If yes -> R should not be a member (by definition). □
If no -> R is exactly the kind of set that belongs in R. □

Resolution: type theory. Separate the levels.
Sets of objects. Sets of sets. Sets of sets of sets.
Each level can only refer to levels below it.
Higher cost. More careful bookkeeping. No paradox.

Mode 4: No seed (Yablo)

PROFESSOR LOGIC

In 1993, Stephen Yablo constructed a paradox with no self-reference at all. A sequence of sentences: sentence 1 says 'all sentences after me are false.' Sentence 2 says the same about everything after sentence 2. And so on, forever. To evaluate sentence 1 you need sentence 2. To evaluate sentence 2 you need sentence 3. There is no starting point. No base case. No seed.

Mode 4: no seed state

S1: 'All Sk for k > 1 are false.'
S2: 'All Sk for k > 2 are false.'
S3: 'All Sk for k > 3 are false.'
...
To evaluate S1 you need S2. To evaluate S2 you need S3. No starting point.
The construction has no base case. It cannot be seeded.
No self-reference. Mode 4 is structurally distinct from Mode 1.

NEW INSIGHT: CONSCIOUSNESS AS APPROXIMATE MODE 3

Russell's paradox shows that a gradient cannot operate at its own definitional level without load overflow. This raises a question.

A conscious mind models itself as part of its context. It thinks about its own thinking. This is self-reference at the same level. By the Mode 3 analysis, the load should overflow. But minds do not collapse.

The resolution: the self-model is approximate. A mind does not contain a perfect replica of itself -- that would be Russell's paradox in neural tissue. It contains a compressed, lossy representation: accurate enough to be useful, inaccurate enough to avoid collapse.

Consciousness is Mode 3 that does not collapse because the self-model is lossy. Perfect self-knowledge would be a paradox, not a goal. This is not a philosophical observation. It is an arithmetic consequence of the load rules applied to self-referential structures.

The four modes at a glance

Summary

Mode Name Core failure Classic example Resolution
1 Liar NOT has no fixed point in V This sentence... Extend V
2 Godel Self-referential load diverges Godel sentence Cannot resolve
3 Russell Gradient demands larger context Russell's set Type stratification
4 Yablo No seed state Yablo's sequence Cannot resolve

CHAPTER 7 EXERCISES

7.1 Solve v = NOT(v) in each carrier: (a) {0,1} (b) {0, 0.5, 1} (c) {0, 0.25, 0.5, 0.75, 1}.

Answer:

(a) No solution: v=0.5 is required but not in the carrier.

(b) Solution: v=0.5.

(c) Solution: v=0.5 (present in this carrier).

7.2 Classify each as Mode 1, 2, 3, or 4. Justify your answer.

(a) "The next sentence is true. The previous sentence is false."

(b) The set of all ordinals (the Burali-Forti paradox).

(c) A program that checks whether it would loop forever.

Answer:

(a) Mode 1: two sentences creating a mutual flip-flop. No fixed point.

(b) Mode 3: the collection of all ordinals would itself be an ordinal. Level collapse.

(c) Mode 2: a self-referential decision procedure. Load grows without bound.

□□□ 7.3 Design a carrier that resolves Mode 1 (the Liar) but still exhibits Mode 2 (load divergence). Explain why resolving M

Answer:

Carrier {0, 0.5, 1}: Mode 1 resolved (v=0.5 is the fixed point). Mode 2 still holds because Mode 2 is about load growth, not about the value space. Self-referential load grows as 2^n regardless of whether 0.5 is available. The two modes are orthogonal.

CHAPTER 8

One Mechanism, Many Faces

Calculus, probability, and modal logic are all the same system in different clothes.

PROFESSOR LOGIC

I want to show you something that surprises most people the first time they see it. And I want to be clear that it is not an analogy. Not 'logic is kind of like calculus.' I mean structurally identical. The same arithmetic. Different value spaces.

Calculus is logic with V = real numbers

AND load rule = Leibniz product rule

Set V = all real numbers. Change the gradient family to differential operations.
The load of a pattern P becomes its derivative: load(P) = dP/dx

Now apply the AND load rule to two patterns P and Q:
load(AND(P, Q)) = load(P) x value(Q) + value(P) x load(Q)

This is identical to the product rule of calculus:
d/dx [f(x) x g(x)] = f'(x) x g(x) + f(x) x g'(x)

The load rule for AND on V = real numbers IS the product rule.
Not an analogy. The Fundamental Theorem of Calculus (integration and differentiation are inverses) is Double Negation in the calculus carrier.

Probability is logic with normalisation

Kolmogorov's axioms as forced laws

Set V = [0, 1]. Interpret values as probabilities.
Add one constraint: probabilities of mutually exclusive events must sum to 1.

The forced laws of this carrier are Kolmogorov's axioms of probability.
P(A) >= 0 -- values are non-negative (V = [0,1] enforces this)
P(certain event) = 1 -- max(V) = 1
P(A or B) = P(A) + P(B) -- from the normalisation constraint

Shannon entropy = the load of a probability distribution.
Maximum entropy = maximum load = most spread-out distribution.

Modal logic is logic with worlds

PROFESSOR LOGIC

Modal logic adds two operators: box (necessarily) and diamond (possibly). In carrier terms: the value space extends to include multiple worlds, and the gradient family gains an accessibility relation between worlds. Which worlds can see which other worlds determines which modal logic you get. S4 logic: accessibility is reflexive and transitive. S5 logic: every world can see every other world. These are carrier choices. Different accessibility choices force different modal theorems.

World logics: other ways to choose V

NEW INSIGHT: WORLD LOGICS AS CARRIER CHOICES

Western logic is not the only formal system tradition. Several traditions independently built carriers that Western logic did not -- and for good reasons rooted in the domains they were reasoning about.

Buddhist catuskoti (four corners):
V = {True, False, Both, Neither}.
Classical logic forces a choice between two. Nagarjuna argued that for claims about ultimate reality, the binary carrier produces systematic misrepresentation.
LNC fails at 'Both'. LEM fails at 'Neither'. Not mistakes. Forced by the carrier.

Taoist logic (the continuous carrier):
V = [0, 1] continuous, with the constraint that v and NOT(v) are always co-present. No state is pure. Stability is dynamic balance, not static fixation.
The natural gradient follows the path of least resistance (wu-wei).
Better matched to ecological systems, relationship dynamics, economic cycles.

Ubuntu (the relational carrier):
'I am because we are.' A person is a person through other persons.
V = relations, not individuals. There are no isolated values.
L = 0 individuals do not exist. Every self is a loaded history of relation.
The Western carrier's isolated individual is the assumption that fails in social and ecological reasoning.

These are not cultural preferences. They are carrier choices made to solve problems that the binary carrier cannot solve.
The laws that follow are forced by the carrier, in every case.

"Classical logic, fuzzy logic, calculus, probability, and the world logics are all one operator -- P / G -> Q -- applied to different value spaces. The diversity is in the carrier. The mechanism is singular."

CHAPTER 8 EXERCISES

8.1 In the calculus carrier (V = real numbers, load = derivative), what is the load of a constant function f(x) = 7?

Answer:

The derivative of f(x)=7 is 0. Load = 0. Constants carry zero load: they have no dependence on the variable. In DRAS, a zero-load quantity is scale-independent. Any quantity with non-zero load varies with context.

8.2 Compute Shannon entropy H = -sum of pi x log2(pi) for: (a) a fair coin, (b) a biased coin (p=0.9, 0.1), (c) a certain

Answer:

(a) H = 1 bit. (b) H approximately 0.469 bits. (c) H = 0 bits.

The fair coin carries the most load. Maximum uncertainty = maximum load.

8.3 Which of the three world logics (catuskoti, Taoist, Ubuntu) uses the largest value space? Which is closest to fuzzy

Answer:

Largest V: catuskoti ({T, F, B, N}) is discrete four-valued; Taoist is continuous [0,1], so largest if continuous. Closest to fuzzy logic: Taoist (continuous V with balance constraint). Closest to relational database theory: Ubuntu (V = relations, not individuals -- this is exactly how relational algebra is structured).

8.4 A sceptic says: "These are just analogies. The actual mathematics is completely different in each case." Write a 15

Answer:

A mere analogy maps concepts to concepts loosely. A structural identity maps operations to operations precisely. The test: write down the AND load rule in carrier-set terms. Write down the product rule in calculus terms. They are the same formula with the same variables in the same positions. No interpretation is required at each step. $\mathrm{load}(\mathrm{AND}(P,Q)) = \mathrm{load}(P) \times \mathrm{value}(Q) + \mathrm{value}(P) \times \mathrm{load}(Q)$. *And* $d/dx[f \times g] = f'g + fg'$. *Same arithmetic. Same structure. The evidence for structural identity: a single unified proof derives both as consequences of the same mechanism on different value spaces. That proof exists and runs in pl_unified.py.*

CHAPTER 9

The Physical Logic

The cube is honest about its carrier.

PROFESSOR LOGIC

I want to show you a formal system you can hold in your hands. Not a diagram of one. Not a description of one. The actual thing, physical and undeniable. The Rubik's cube was invented in 1974 by Erno Rubik to help students understand three-dimensional space. He accidentally built the world's best teaching tool for formal system mechanics.

The cube is a carrier

The Rubik's cube: (V, G, theta)

V -- the value space:
    Every possible arrangement of the 20 moveable pieces.
    43,252,003,274,489,856,000 possible values.
    You hold V in your hands.

G -- the gradient family:
    Exactly 18 moves: U U2 U' D D2 D' F F2 F' B B2 B' R R2 R' L L2 L'
    Fixed by physical design. Cannot be extended.
    The plastic will not rotate any other way.

theta -- the coherence threshold:
    God's number = 20. Every state in V is reachable from solved in 20 moves.
    The physical object enforces theta. You cannot exceed it.

L -- the load:
    L(state) = minimum moves to reach the solved (seed) state.
    L(solved) = 0. The seed state. L(worst case) = 20 = theta.

The forced laws

The group-theoretic laws are not rules someone imposed on the cube. They fell out of the physical design.

Forced laws of the cube carrier

Closure: Any move applied to any state gives another valid cube state.
You cannot leave V. The cube stays a cube.
Inverses exist: Every move has an undo. U' undoes U. This makes solving possible.
Identity: The solved state exists. L = 0.
Associativity: (U then R) then F = U then (R then F). Grouping does not matter.
G^4 = identity: Rotate any face four times and you are back to start.
Half-turn: U2 applied twice returns to start. NOT(NOT(v)) = v in plastic.
Non-commutativity: U then R is NOT the same as R then U. Order matters.

What the carrier forbids

PROFESSOR LOGIC

Some apparent moves are not in G. They are not illegal by rule. They are incoherent in this carrier. You cannot swap two corners without affecting other pieces. You cannot flip a single edge. Take a corner off and put it back rotated 120 degrees. Now you have a state that is physically a valid arrangement of plastic but formally outside the carrier reachable from the solved state. Try to solve it. You cannot -- not because it is far away, but because it is outside the group. This is the most important moment in the lesson. The cube makes you hold the impossible state in your hands.

The pedagogical map

Cube concept <--> Formal system concept

All sticker positions V: the value space
Solved state Seed state: L = 0
Scrambled state Loaded pattern: L > 0
Face rotations G: the gradient family
God's number (20) theta: the coherence threshold
G^4 = identity Forced law: each gradient has order 4
Half-turn applied twice NOT(NOT(v)) = v. Double negation in plastic.
U then R not equal R then U Order matters in real systems
Subgroup structure Carriers within carriers
Disassembled cube State outside the reachable carrier (incoherent, not hard)

The six-step lesson

Step 1. Hold the solved cube. This is the seed state. L = 0. The centres never move relative to each other. This is a forced law, not a design choice.

Step 2. Make one move. You just applied P / G -> Q. L went from 0 to 1.

Step 3. Scramble it. L is now between 1 and 20. Every position is reachable from solved. This is what a coherent but loaded formal system looks like.

Step 4. Try to swap two adjacent corners without anything else moving. You cannot. Not a rule. A physical incoherence. This is the key moment.

Step 5. Take a corner off. Put it back rotated. You now have a state outside the reachable carrier. Try to solve it. You cannot.

Step 6. Ask: what formal systems in your life might be doing this? What arguments cannot be resolved because they start from a position outside the reachable carrier?

CHAPTER 9 EXERCISES

9.1 The cube has exactly 18 generators (moves). How many are their own inverse? How many have order 4?

Answer:

Own inverse (order 2): the six half-turns U2, D2, F2, B2, R2, L2. Each applied twice returns to identity.

Order 4: the six clockwise and six counter-clockwise moves (U, U', D, D', F, F', B, B', R, R', L, L'). 12 moves of order 4.

9.2 Why is the disassembled cube the most important pedagogical moment in Chapter 9?

Answer:

Because it makes incoherence physical. In abstract formal systems, states outside the reachable carrier look like states inside it. They look like valid moves, valid arguments, valid premises. The cube forces you to hold the impossible state in your hands and feel that no sequence of legal moves will ever fix it. That is irreplaceable. The distinction between "hard to reach" and "outside the carrier entirely" is the most important distinction in formal system engineering.

□□□ 9.3 The half-turn subgroup consists of just the six half-turns {U2, D2, F2, B2, R2, L2}. Every element has order 2: it is i

Answer:

Every element is self-inverse: G = G^-1. This means the group is abelian (commutative): U2 then D2 = D2 then U2 in this subgroup. Double Negation holds for all elements. The subgroup is isomorphic to Z2 x Z2 x Z2 x Z2 x Z2 x Z2, a direct product of six copies of the simplest non-trivial group. All elements have the same status: no element generates a richer structure than any other.

CHAPTER 10

Language as a Carrier

We have been using language to describe formal systems. Language is a formal system.

PROFESSOR LOGIC

We need to turn the framework back on itself. For nine chapters I have been using language to describe carriers, operations, and forced laws. I have been using the subject-predicate structure of English to say things like 'LNC is forced in {0,1}.' Every sentence I have written is itself an operation on a value space. What is the carrier of natural language? What are its operations? What are its forced laws? These are not rhetorical questions. They have specific answers.

The subject-predicate cut is P / G -> Q

Every declarative sentence has the form:

```
[Subject] [Predicate] [Truth value]
     P         G            Q
```

'The cat is on the mat' = TRUE or FALSE
'The water is boiling' = TRUE or FALSE
'The policy is working' = ??? (the carrier is unclear)

The subject is the pattern being evaluated.
The predicate is the gradient applied to it.
Grammar is the gradient family: rules for which subjects can combine with which predicates to form valid sentences.

PROFESSOR LOGIC

Noam Chomsky wrote a famous sentence: 'Colorless green ideas sleep furiously.' It is grammatically impeccable and semantically absurd. In carrier-set terms: the sentence is a valid operation on the grammatical gradient family. But the value it tries to express is not in the semantic value space. 'Colorless green' demands something both colourless and green simultaneously. The grammatical operations produced a result outside the semantic carrier. This is a closure failure in the semantic carrier, not the grammatical one.

What is the carrier of natural language?

Classical logic claims vs. natural language reality

Classical logic claims: the carrier of all statements is {TRUE, FALSE}.

Natural language actually uses something more like:
V = { FALSE, UNLIKELY, POSSIBLE, PROBABLE, TRUE,
ALLEGEDLY, SUPPOSEDLY, IRONICALLY TRUE,
METAPHORICALLY TRUE, TRUE IN THIS CONTEXT, ... }
Plus modifiers from tense: past, present, future, conditional, subjunctive.

This is not a two-value carrier. It is a high-dimensional continuous space with a few discrete landmarks, navigated at the speed of speech.

The sorites paradox: Mode 1 in language

The sorites paradox (from Greek: soros = heap)

1,000,000 grains of sand is a heap. TRUE.
If n grains is a heap, then n-1 grains is also a heap. TRUE? (plausibly)
Therefore: 1 grain of sand is a heap. FALSE.

The predicate 'is a heap' demands a sharp boundary in the carrier.
No sharp boundary exists in the physical world.
The predicate is trying to apply a {0,1} operation to a continuous reality.
This is Mode 1: the predicate demands a value not in the carrier.

Resolution attempt: extend V to include 'borderline heap' = 0.5.
But then: is 999,998 grains a borderline heap or a borderline-borderline heap?
Higher-order vagueness. Mode 2 entering language: load diverges.

"Language is logic with the debt hidden. Formal logic is language with the debt made explicit. Poetry is language that celebrates the fact that the debt can never be fully paid."

CHAPTER 10 EXERCISES

10.1 Take "The soup is hot." Identify the subject, predicate, and implicit carrier. What information is lost if you force it int

Answer:

Subject: "the soup." Predicate: "is hot." Carrier: not {TRUE, FALSE}. Hot is relative to the speaker's preference, the intended use, and the context. Forcing to {TRUE, FALSE} loses all scalar and contextual information.

10.2 The sorites paradox. Which paradox mode is it? Propose a minimal extension to the language carrier that resolves

Answer:

Mode 1: the predicate demands a boundary value not in the carrier. Resolution attempt: extend V to {0, 0.5, 1}. Mode 2 follows: what is a borderline-borderline heap? Each level of qualification requires the next. Load grows without limit at each level of meta-qualification.

10.3 Consider "I am lying right now" spoken aloud. Map it onto the carrier-set framework. Does it resolve differently in

Answer:

Primary: Mode 1 (the Liar in natural language). But spoken language adds a temporal dimension: "now" is a specific moment. The self-reference always occurs across moments. The paradox dissolves into Mode 2 (infinite regress) rather than a static contradiction. Written text has no such temporal relief. Each evaluation step occupies a new moment in spoken language.

CHAPTER 11

Coding a Carrier

If you can write it, you understand it. Thirty lines of Python.

PROFESSOR LOGIC

Every claim in this book has been mathematical. Now I want to make it concrete in a different way: code that you can run, modify, and break. This chapter requires Python 3. No installation of any library. Just Python and a text editor or the Python shell. The goal is simple: by the end of this chapter you will have a Carrier class that can test any carrier you give it and report which laws hold.

Step 1: the value space as a list

PYTHON

```python
# A list in Python is written with square brackets
V = [0, 1] # classical carrier
V = [0, 0.5, 1] # three-valued carrier
V = [i/100 for i in range(101)] # 101 values from 0 to 1
```

Step 2: the operations as functions

PYTHON

```python
# lambda is Python for 'a small function with no name'
NOT = lambda v: 1 - v
AND = lambda a, b: a * b
OR = lambda a, b: max(a, b)
```

Step 3: checking a law

```python
def check_lnc(V, AND, NOT):
    return all( abs(AND(v, NOT(v))) < 1e-10 for v in V )

def check_lem(V, OR, NOT):
    top = max(V)
    return all( abs(OR(v, NOT(v)) - top) < 1e-10 for v in V )

def check_dn(V, NOT):
    return all( abs(NOT(NOT(v)) - v) < 1e-10 for v in V )

V = [0, 1]
NOT = lambda v: 1 - v
AND = lambda a, b: a * b
OR = lambda a, b: max(a, b)
print(check_lnc(V, AND, NOT)) # True
print(check_lem(V, OR, NOT)) # True
print(check_dn(V, NOT)) # True
```

Step 4: finding the Liar solution

```python
def liar_solutions(V, NOT, tol=1e-10):
    return [v for v in V if abs(v - NOT(v)) < tol]

# Classical carrier: no solution
print(liar_solutions([0, 1], lambda v: 1-v)) # []

# Three-valued carrier: one solution
print(liar_solutions([0, 0.5, 1], lambda v: 1-v)) # [0.5]
```

Step 5: the complete Carrier class

PYTHON

```
class Carrier:
    def __init__(self, V, not_, and_, or_, theta=1.0):
        self.V = V
        self.not_ = not_
        self.and_ = and_
        self.or_ = or_
        self.theta = theta

    def lnc(self):
        return all(abs(self.and_(v, self.not_(v))) < 1e-10
                   for v in self.V)

    def lem(self):
        top = max(self.V)
        return all(abs(self.or_(v, self.not_(v)) - top) < 1e-10
                   for v in self.V)

    def dn(self):
        return all(abs(self.not_(self.not_(v)) - v) < 1e-10
                   for v in self.V)

    def liar(self):
        return [v for v in self.V
                if abs(v - self.not_(v)) < 1e-10]

    def report(self):
        print(f'V = {self.V[:6]}')
        for name, fn in [('LNC',self.lnc),('LEM',self.lem),('DN',self.dn)]:
            mark = 'correct' if fn() else 'fails'
            print(f' {name}: {mark}')
        sols = self.liar()
        if sols: print(f' Liar resolved at {sols}')
        else: print(' Liar: paradox (no fixed point)')

c = Carrier([0,0.5,1], lambda v:1-v, lambda a,b:a*b, lambda a,b:max(a,b))
c.report()
```

CHAPTER 11 EXERCISES

11.1 Run the Carrier class on the classical {0,1} and three-valued {0,0.5,1} carriers. Verify the output matches your hand c

□□ 11.2 Add a closure_check method that tests whether NOT maps every value in V back to a value in V. Test it on the

Answer:

def closure_check(self): V_set = set(round(v,10) for v in self.V); return all(round(self.not_(v),10) in V_set for v in self.V). A NOT that returns 2.0 for value 1 will fail: 2.0 is not in {0, 0.5, 1}.

□□ 11.3 Implement the generalised AND rule. Add a method and_load(LP, LQ, mutual_info) that returns LP + LQ -

Answer:

def and_load(self, LP, LQ, I=0): return LP + LQ - I. With I=0: returns LP + LQ. With I=LP: returns LQ. The second operand is free given the first.

□□□ 11.4 Write a compare(c1, c2) function that prints a table showing which laws each carrier has in common and which

Answer:

```
def compare(c1, c2):
checks = [('LNC',lambda c:c.lnc()),('LEM',lambda c:c.lem()),
('DN',lambda c:c.dn()),('Liar',lambda c:bool(c.liar()))]
for name,fn in checks:
r1,r2=fn(c1),fn(c2)
changed=' <-- CHANGED' if r1!=r2 else ''
print(f'{name}: {r1} vs {r2}{changed}')
```

CHAPTER 12

Your Challenge

Design a formal system. Verify it. Defend it.

PROFESSOR LOGIC

This is the final chapter, and it is the most open. There is no single correct answer. There is only a standard: every claim must be verified, either by hand or by code. If you cannot verify it, it is not finished. That standard is the empirical standard. It is what separates a formal system from a philosophical opinion.

The requirements

1. Carrier specification
   State V. State your encoding choices. State which values are designated.
2. Operations
   Define NOT, AND, OR for every value. Prove closure.
3. Forced laws
   Check LNC, LEM, DN. State at least two additional laws your carrier forces.
   State at least two laws that are NOT forced, with counterexamples.
4. Load and threshold
   State theta. Give one example pattern that coheres and one that does not.
5. Failure modes
   State two situations where your system breaks. Name the mode (1, 2, 3, or 4).
6. Utility
   When is this logic better than classical {0,1}?
   What does it model that classical logic cannot?
7. Verification (required)
   Either hand-verify all claims, or run the Carrier class from Chapter 11.

Suggested domains

Domains that reward non-classical carriers

Medical diagnosis Partial evidence, competing hypotheses, threshold effects
Environmental risk Continuous values, irreversibility, multi-scale measurements
Legal evidence Weight of evidence, burden of proof, categorical distinctions
Financial modelling Probability over outcomes, resource constraints, time
Social trust Relational values, transitivity, history-dependence
Game states Partial information, simultaneous action, payoff structure
Ecological capacity Resource limits, thresholds, multi-species interaction

PROFESSOR LOGIC

One final note on the distinction operation idea I raised in the preface. A distinction operation D(a, b) would measure the cost of telling two values apart. In a carrier where all values are equally spaced, D might simply be the absolute difference |a - b|. In a carrier with non-uniform spacing, the cost of distinguishing nearby values might be higher. D(0.49, 0.51) should cost more than D(0, 1). A formal distinction operation could be D(a, b) = |a - b| divided by the range of V: a normalised distance in the value space. I leave this as an open thread. If you formalise it in your Chapter 12 project, send it along. Good ideas deserve company.

"Every formal system is a choice. The choice of what values to allow. The choice of how to combine them. The choice of how much to tolerate. Change any one and a different world of consequences becomes forced. Make your choices deliberately. Verify what they force. That is the whole of the discipline."

ANSWER KEY

Selected Answers

PROFESSOR LOGIC

Every chapter exercise has a full answer embedded inline. This section summarises the core results for quick reference when reviewing.

Law results across carriers

Law {0,1} {0,0.5,1} [0,1] sampled
Non-Contradiction forced fails at 0.5 fails at interior
Excluded Middle forced fails at 0.5 fails at interior
Double Negation forced forced forced
Liar solution none v = 0.5 v = 0.5

Core formulas

NOT(v) = 1 - v AND(a,b) = a x b OR(a,b) = max(a,b)

Load rules:
NOT: unchanged AND: adds OR: takes minimum

Generalised AND: load = LP + LQ - I(P,Q) Standard: I = 0

Full load spectrum:
L < 0: tautology L = 0: seed 0 < L < theta: coherent L > theta: overflow

Liar fixed point: v = NOT(v) --> v = 1-v --> 2v = 1 --> v = 0.5

Three parameters: V (values) G (operations) theta (threshold)

Four paradox modes: Liar Godel Russell Yablo

A = A hidden assumptions: Absoluteness, Temporal Stability, Context Independence

Newton's theta: load of Newtonian velocity assumption approximately (v/c)^2